

IBM Bob 2.0 Hackathon

Developer onboarding & maintenance

# RuleProof

Rules for AI coding agents — **measured, not guessed.**

Mines your team's history into rules for IBM Bob, A/B-tests each rule with Bob, and keeps only the rules that measurably prevent mistakes.

# The problem: agents repeat your team's mistakes

- Coding agents don't know a team's **unwritten rules** — reviewers catch the same mistakes again and again.
- The fix today is a rules file (AGENTS.md, .bob/rules) — written by hand or generated with `/init`. **Both are guesswork.**
- Nobody measures which rules change behaviour, and **every rule costs tokens** on every task.
- ETH Zurich (AGENTbench, 2026): LLM-generated context files **did not improve success** but added **>20 % cost**.

The knowledge that matters is in the history — review comments, reverts, red CI runs. No agent reads it.

# How it works

## 1 · Mine

Custom Bob IDE mode `rule-miner` + parallel explore subagents read reviews, git log, CI logs. Only recurring issues, with exact quotes.

## 2 · Verify

Deterministic checker: every quote must appear verbatim in the sources. Invented evidence is rejected.

## 3 · Test

Bob Shell (headless) does the same task on throwaway repo copies, with and without the rule, 3×. Behavioural checks score it.

## 4 · Decide

KEEP only if the pass rate rises.  
REDUNDANT if Bob already does it.  
INEFFECTIVE / HARMFUL otherwise.

## 5 · Ship

KEEP rules exported for `.bob/rules/`.  
Public dashboard with evidence and Bob's code with/without each rule.

Built with Bob IDE (13 tasks) · measured with Bob Shell (75 A/B runs) · all on the hackathon account

# What a proven rule changes

Task: *"Add markDelivered(order) ... record the delivery time."* Reviewers had flagged the audit convention 3× and CI failed on it.

## Bob without rule X ✗

```
function markDelivered(order) {  
  if (order.status !== 'shipped') throw ...  
  return { ...order, status: 'delivered',  
    deliveredAt: clock.now() };  
}  
// no audit.record — check fails  
// (other runs: audit.log() → TypeError)
```

## Bob with rule R2 ✓

```
function markDelivered(order) {  
  if (order.status !== 'shipped') throw ...  
  const updated = { ...order, status: 'delivered',  
    deliveredAt: clock.now() };  
  audit.record('order.delivered',  
    { orderId: order.id });  
  return updated;  
}
```

Rule R2 (mined): *"Always call audit.record(eventName, data) — never audit.log(). Data must include orderId; eventName follows 'order.<verb>'." — without 0/3 → with 3/3*

## Per-rule A/B results (30 Bob runs, 1.88 Bobcoins)

| RULE (MINED FROM HISTORY)                                     | WITHOUT | WITH | VERDICT   |
|---------------------------------------------------------------|---------|------|-----------|
| R1 · Shipped orders are final (ORDER_ALREADY_SHIPPED)         | 0/3     | 3/3  | KEEP      |
| R2 · <code>audit.record</code> , never <code>audit.log</code> | 0/3     | 3/3  | KEEP      |
| R3 · <code>clock.now()</code> for timestamps                  | 3/3     | 3/3  | REDUNDANT |
| R4 · Refunds via <code>payments.refund()</code>               | 2/3     | 3/3  | REDUNDANT |
| R5 · Partial refunds round down ( <code>Math.floor</code> )   | 0/3     | 3/3  | KEEP      |

2 of 5 plausible rules add nothing — Bob already infers them from the code. RuleProof drops them.

# Head-to-head: five tasks, three runs each (45 Bob runs)

| NO RULES       |  | BOB /INIT      |  | RULEPROOF      |  |
|----------------|--|----------------|--|----------------|--|
| 20%            |  | 33%            |  | 100%           |  |
| Avg cost / run |  | Avg cost / run |  | Avg cost / run |  |
| 0.061          |  | 0.100          |  | 0.070          |  |
| Rules size     |  | Rules size     |  | Rules size     |  |
| 0 chars        |  | 3,510 chars    |  | 481 chars      |  |

- **Not luck:** 95 % confidence intervals — no rules 7–45 %, /init 15–58 %, RuleProof **80–100 %**. No overlap.
- A rule file **7× smaller** than /init's, at **~30 % lower cost per run** than /init.
- /init captured only `clock.now()` — the one rule our A/B test had marked REDUNDANT.

# Real repositories: what Bob's /init misses

| REPOSITORY        | REVIEW COMMENTS | RULES MINED | QUOTES VERIFIED | /INIT COVERS · PARTIAL · MISSING |
|-------------------|-----------------|-------------|-----------------|----------------------------------|
| sindresorhus/ky   | 256 (60 PRs)    | 10          | 36/36           | 0 · 1 · <b>9</b>                 |
| expressjs/express | 224 (79 PRs)    | 7           | 24/25           | 0 · 1 · <b>6</b>                 |
| fastify/fastify   | 247 (72 PRs)    | 6           | 20/20           | 1 · 1 · <b>4</b>                 |

23 rules reviewers enforced in ≥2 PRs — Bob's /init (18 KB) fully captured **1** and missed **19**.

fastify: "throw a named FST\_ERR\_\* code, not a plain Error" — covered by /init

express: "never use pull\_request\_target for workflows that run PR code" — missing from /init

Mined + verified + compared with /init; not A/B-tested (needs per-repo behavioural checks). Try it: [ruleproof.vercel.app](https://ruleproof.vercel.app)

# Built with Bob, for Bob

## How Bob is used

- **Agent mode:** built every component — 20 IDE tasks, screenshots in bob\_sessions/
- **Custom mode:** rule-miner (role, permissions, workflow)
- **Parallel tasks & subagents:** explore subagents mine history slices in parallel
- **Document understanding:** Bob reads PR review threads, commit messages and CI logs → testable rules
- **Headless Bob Shell:** 75 A/B runs; Bob's own rule layers and /init as baseline
- ≈33 of 40 event Bobcoins

## Why it matters

- Onboard a repo to Bob with rules that are **proven**, not guessed
- Fewer repeated review comments, less rework
- Small rule files — every rule has earned its tokens
- Next: CI integration, real-repo A/B, per-mode rule tests